

ỨNG DỤNG PHÂN TÁN

II. Quản lý tiến trình và luồng

Nhóm chuyên môn UDPT – Trường Công nghệ thông tin Phenikaa
(Trần Đăng Hoan, Đỗ Quốc Trường, Nguyễn Thành Trung, Phạm Kim Thành, Nguyễn Hữu Đạt)

NỘI DUNG

1. Tiến trình và luồng
2. Tiến trình và luồng trong HPT
3. Server trong HPT
4. Client trong HPT
5. Ứng dụng & Bài tập

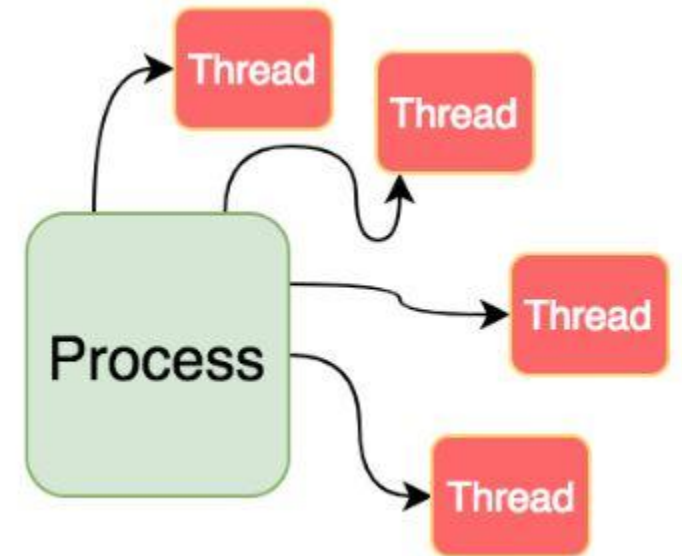
TIẾN TRÌNH VÀ LUỒNG

- Tiến trình (process): Là 1 chương trình đang được chạy trên một trong các bộ xử lý ảo của hệ điều hành.
 - Chương trình đang hoạt động
 - Tài nguyên:
 - Virtual Processor
 - Virtual Memory
 - Trong suốt tương tranh
 - Các tiến trình khác nhau nằm trong một không gian địa chỉ riêng biệt và không truy nhập bộ nhớ của nhau

TIẾN TRÌNH VÀ LUỒNG

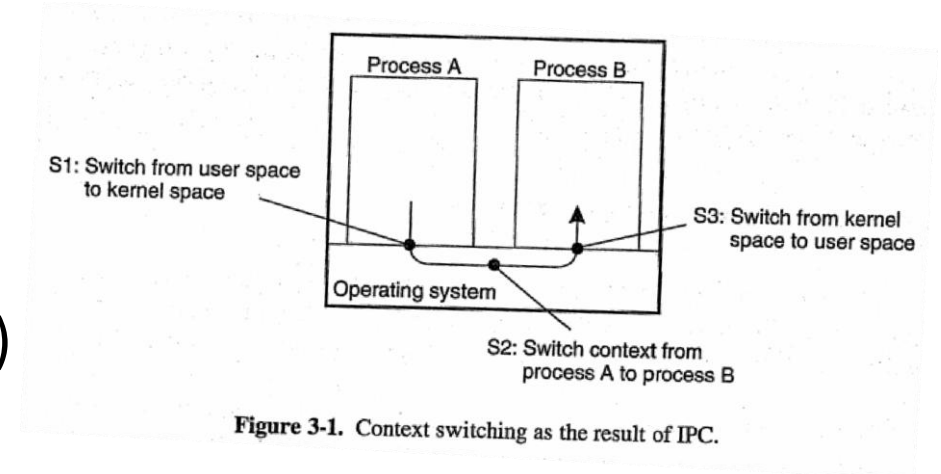
Luồng (thread):

- ❑ Là một luồng thực thi của tiến trình.
- ❑ Tiến trình có nhiều luồng thực thi => Tiến trình đa luồng
- ❑ Các luồng thuộc cùng một tiến trình thì cùng sử dụng không gian bộ nhớ của tiến trình đó
- ❑ Trao đổi thông tin giữa các luồng thông qua các biến chia sẻ
- ❑ An toàn và hợp lý của tương tác luồng do lập trình viên quyết định
- ❑ Luồng => hiệu năng+chi phí lập trình



TIẾN TRÌNH VÀ LUỒNG

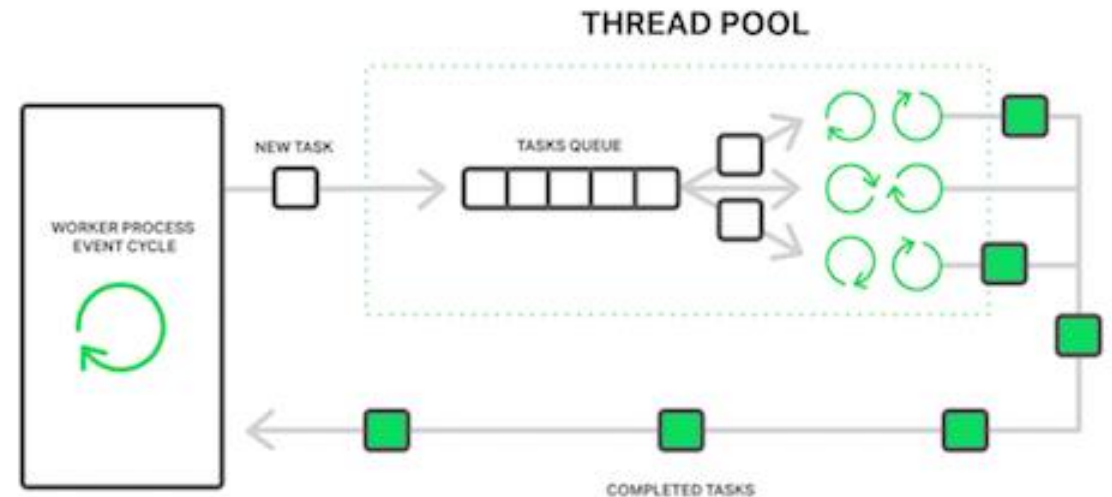
- Khi 1 tiến trình chạy 1 chương trình nó sẽ sinh ra 1 hay nhiều luồng.
- Sau khi được tiến trình sinh ra mỗi luồng có đoạn mã thực hiện riêng, độc lập với các luồng khác.
- Tuy nhiên khác với tiến trình, các luồng không cần phải hoạt động hoàn toàn độc lập và trong suốt với nhau. Vì vậy mỗi luồng chỉ cần lưu trữ thông tin ít nhất để có thể cho phép các luồng chia sẻ CPU
- Đa tiến trình vs. Đa luồng.
 - Chi phí lập trình
 - Chi phí chuyển ngữ cảnh
 - Lời gọi hệ thống dừng (blocking system calls)



TIẾN TRÌNH VÀ LUỒNG

Cách thức cài đặt luồng

- Các luồng thường được cung ứng bởi các gói luồng.
- Các gói bao gồm các nhiệm vụ:
 - Khởi tạo luồng
 - Giải phóng luồng
 - Đồng bộ các luồng
- Có 2 hướng tiếp cận để cài đặt luồng:
 - User mode
 - Kernel mode



TIẾN TRÌNH VÀ LUỒNG

Xây dựng luồng kiểu người dùng (User mode)

- Ưu điểm:
 - Tiết kiệm tài nguyên hệ thống để tạo và hủy luồng. Vì quản lý luồng nằm hoàn toàn ở vùng nhớ của user, chi phí để tạo luồng được xác định bằng chi phí để cấp phát vùng nhớ được tạo 1 stack các luồng
 - Việc chuyển ngữ cảnh được thực hiện nhanh. Thực tế việc chuyển ngữ cảnh được thực hiện khi 2 luồng đồng bộ hóa
- Nhược điểm:
 - Lời triệu gọi của lời gọi hệ thống dừng sẽ làm dừng toàn bộ tiến trình chứa luồng đó và tất cả các luồng khác do tiến trình đó sinh ra

TIẾN TRÌNH VÀ LUỒNG

Xây dựng luồng kiểu nhân (Kernel mode)

- Ưu điểm:
 - Vấn đề bị dừng tiến trình ở trên được giải quyết ở hướng tiếp cận này tuy nhiên chi phí rất lớn.
- Nhược điểm:
 - Mỗi thao tác với luồng như tạo, hủy, đồng bộ sẽ được đảm nhiệm bởi nhân và cần 1 lời gọi hệ thống => gây tốn kém chi phí
 - Việc chuyển ngữ cảnh luồng trở nên tốn kém tài nguyên hệ thống như việc chuyển ngữ cảnh tiến trình => các lợi ích về hiệu năng khi sử dụng đa luồng không còn

TIẾN TRÌNH VÀ LUỒNG

Xây dựng luồng kiểu người dùng (User mode)

X

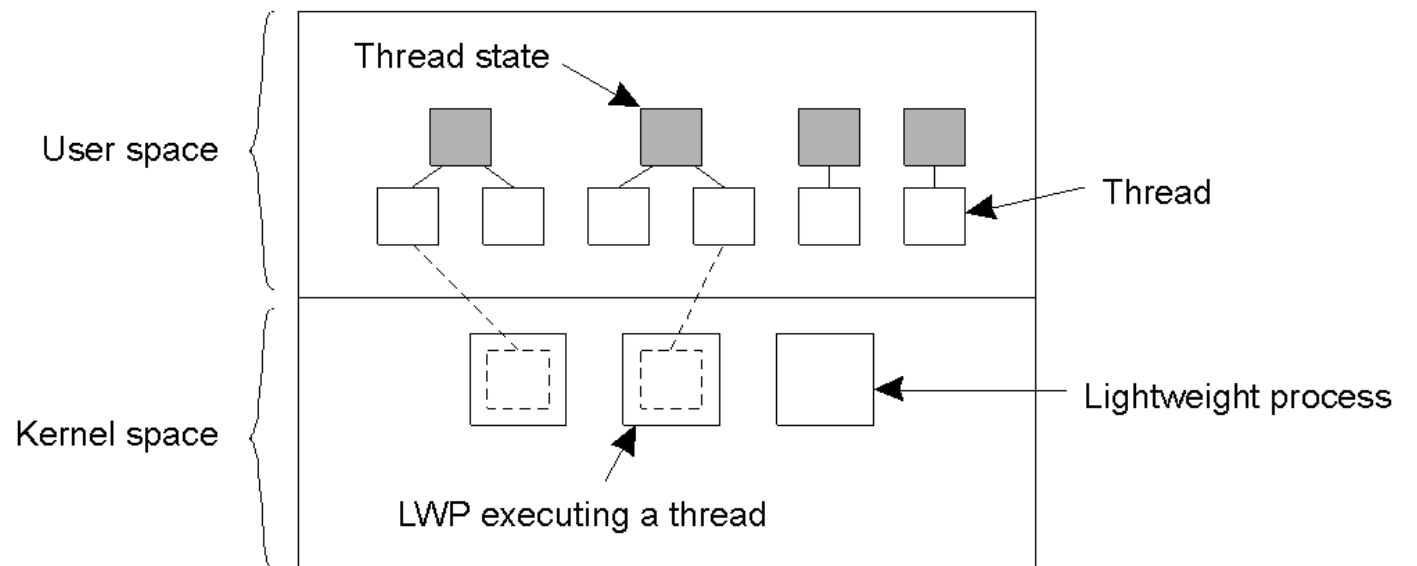
Xây dựng luồng kiểu nhân (Kernel mode)

- ⇒ Hệ thống tận dụng ưu điểm của cả 2 phương pháp này
- ⇒ Tiến trình nhẹ (lightweight processes) được sinh ra

TIẾN TRÌNH VÀ LUỒNG

Tiến trình nhẹ (lightweight processes)

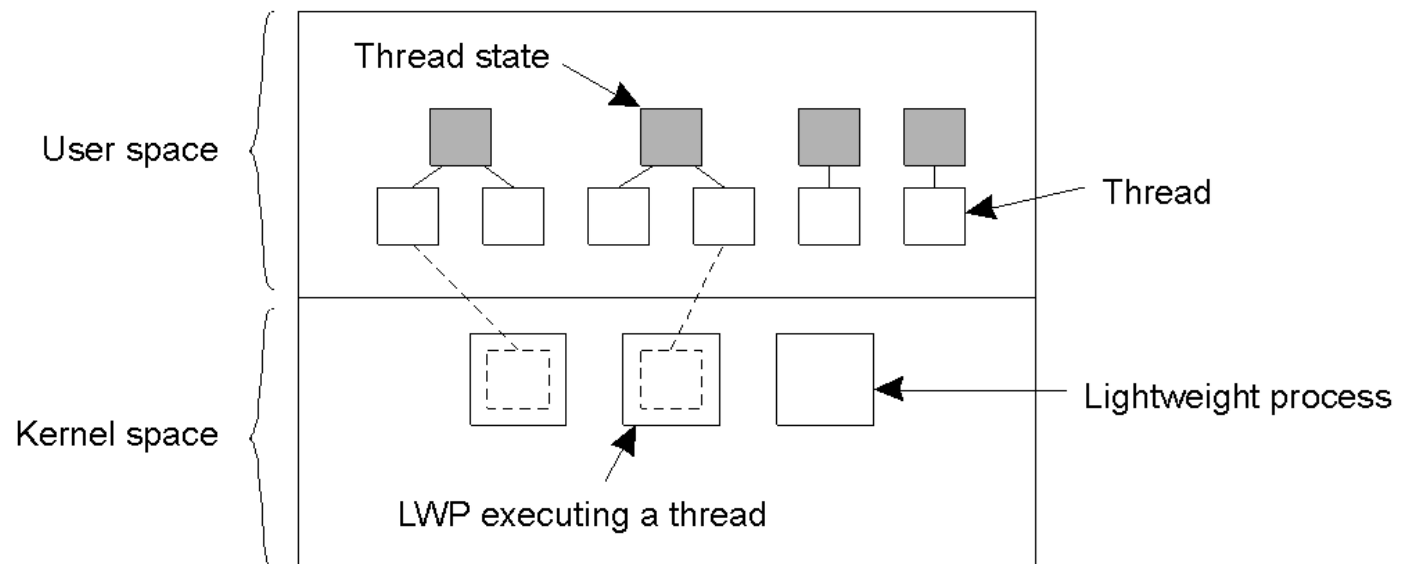
- Khi chạy ngữ cảnh ở 1 tiến trình đơn và mỗi tiến trình đơn có nhiều tiến trình nhẹ.
- Ngoài việc có nhiều tiến trình nhẹ, hệ thống cung cấp gói luồng ở mức user, trợ giúp các ứng dụng về việc tạo và hủy luồng, đồng bộ hoá.
- Vì việc sử dụng gói luồng hoàn toàn ở tầng user do đó các thao tác với luồng không cần can thiệp của tầng kernel => giảm nhiều chi phí



TIẾN TRÌNH VÀ LUỒNG

Tiến trình nhẹ (lightweight processes)

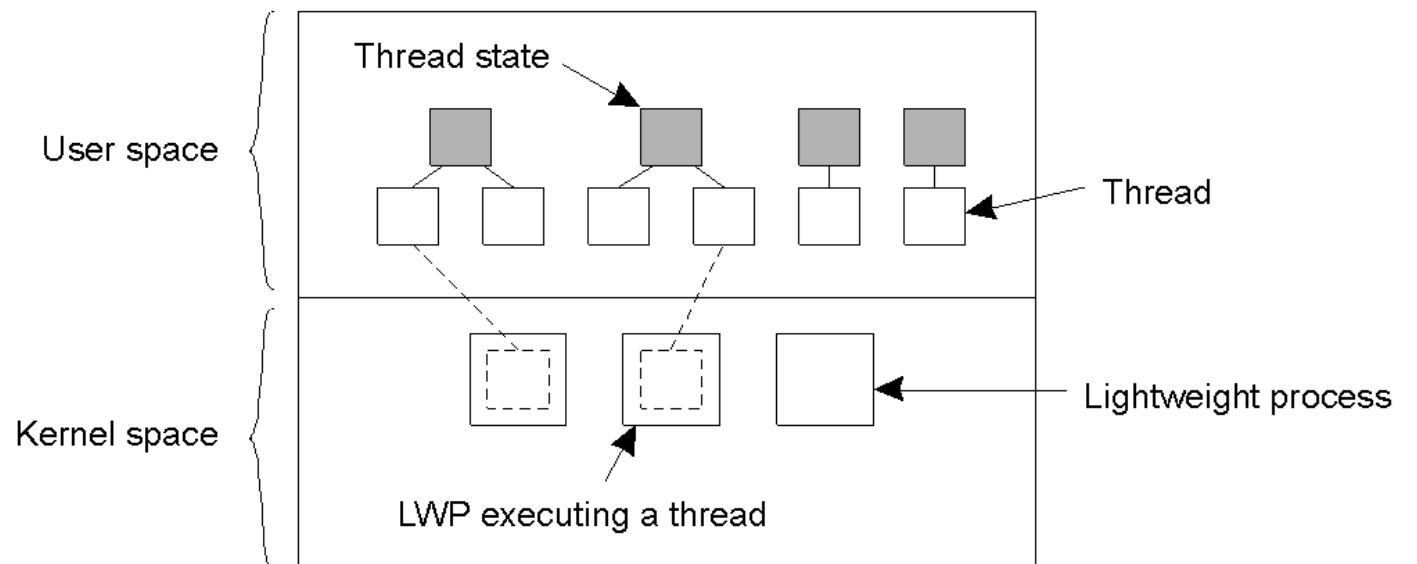
- 1 tiến trình sinh ra nhiều tiến trình nhẹ
- 1 tiến trình nhẹ được gán với nhiều luồng
- Khi 1 luồng có thao tác vào ra tiến trình nhẹ tương ứng bị treo, tuy nhiên các tiến trình nhẹ khác vẫn tiếp tục được thực hiện



TIẾN TRÌNH VÀ LUỒNG

Ưu điểm Tiến trình nhẹ (lightweight processes)

- Các thao tác với luồng có chi phí thấp và hoàn toàn không có sự can thiệp của tầng nhân.
- Khi 1 tiến trình có đủ các tiến trình nhẹ, 1 lời gọi hệ thống dừng không làm dừng cả hệ thống mà chỉ làm dừng 1 tiến trình nhẹ.
- Ứng dụng không cần phải biết về các tiến trình nhẹ, nó chỉ nhìn thấy các luồng ở mức user
- Tiến trình nhẹ có thể được sử dụng trong môi trường đa vi xử lý, mỗi tiến trình nhẹ gán với 1 CPU



LUỒNG TRONG HPT

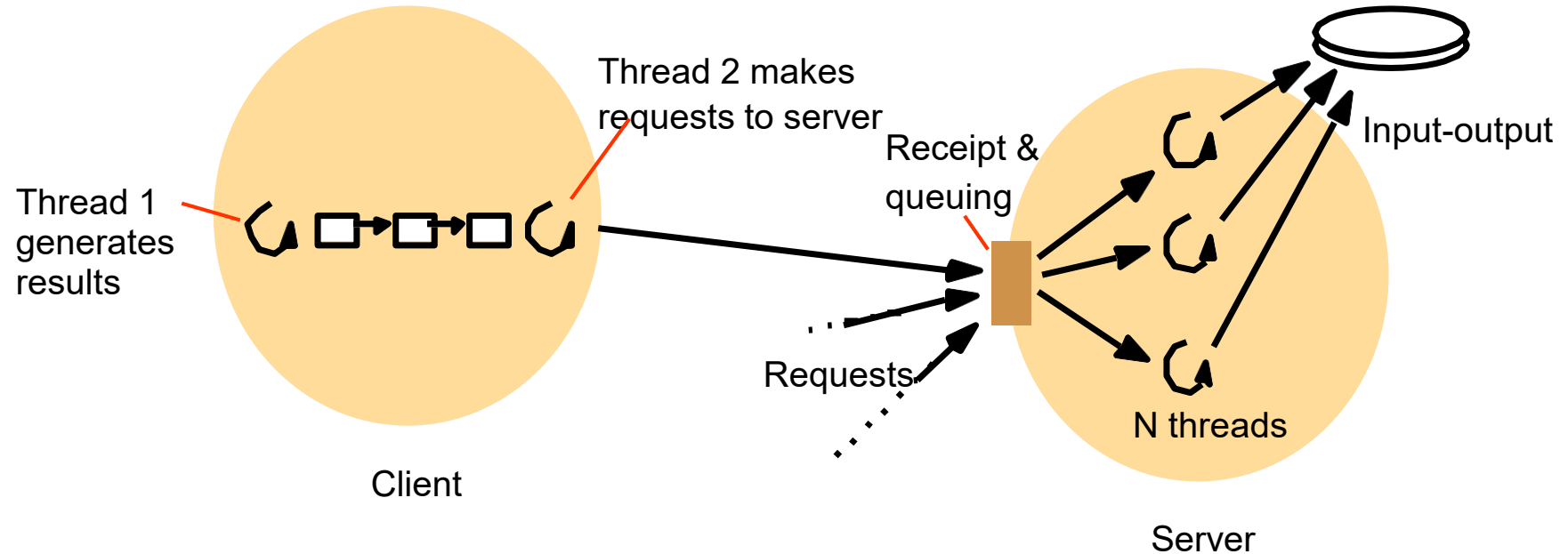
- Server đơn luồng
 - Chỉ xử lý được một yêu cầu tại một thời điểm
 - Các yêu cầu có thể được xử lý tuần tự
 - Các yêu cầu có thể được xử lý bởi các tiến trình khác nhau
 - Không đảm bảo tính trong suốt



Server đa luồng

LUỒNG TRONG HPT

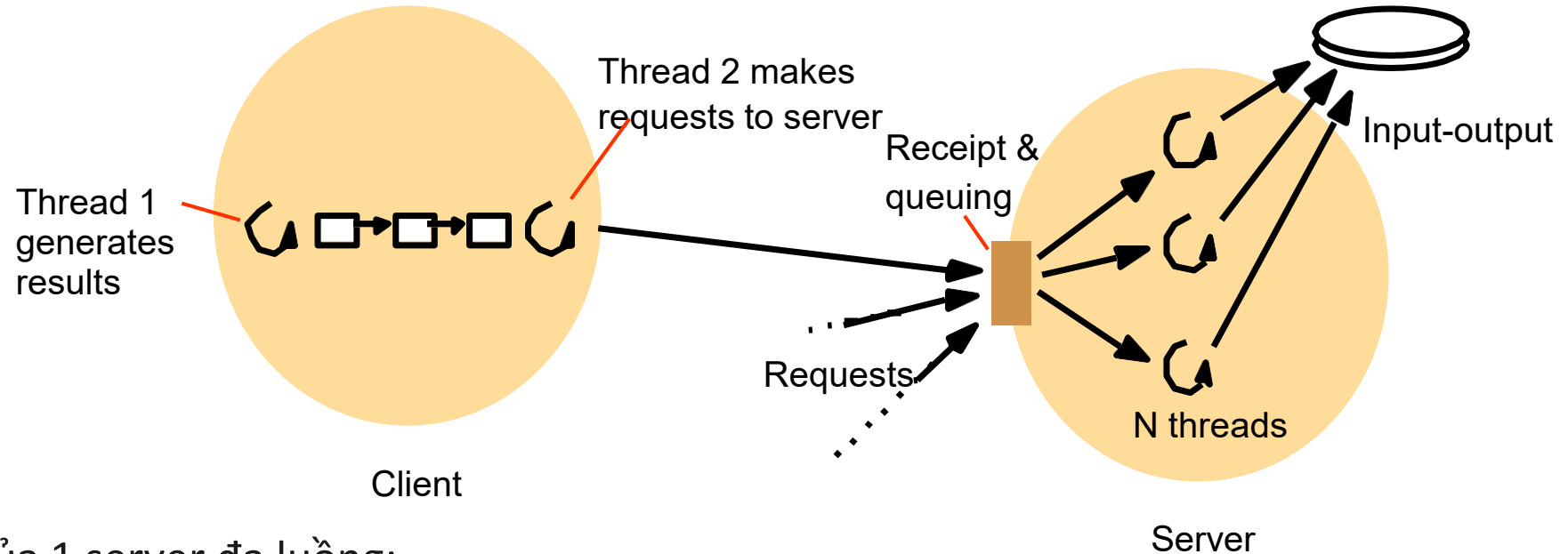
Client và server đa luồng



- Client:
 - 1 luồng chuyên sinh ra các gói tin request
 - 1 luồng khác lo việc đóng gói gói tin và gửi đi cho server
- Server:
 - 1 luồng chuyên lo thu nhận các gói tin và xếp chúng vào hàng đợi.
 - Sau đó gửi chúng đi các luồng khác chuyên xử lý thông điệp. Các luồng này sẽ trực tiếp làm việc với các module vào ra

LUỒNG TRONG HPT

Client và server đa luồng



=> Chức năng chính của 1 server đa luồng:

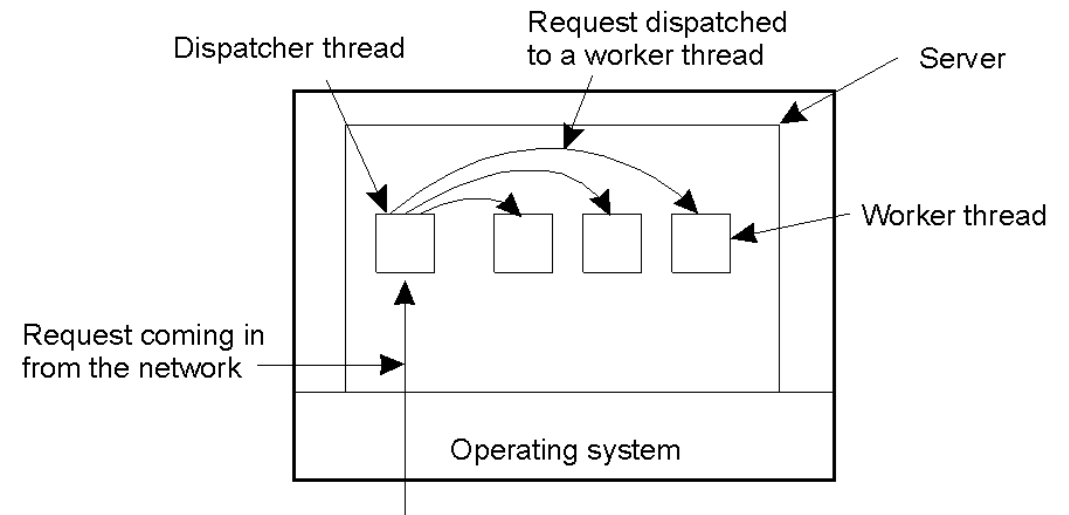
- 1 luồng có nhiệm vụ thu nhận request từ client và gửi vào các luồng xử lý, nếu không có đủ luồng xử lý sẽ xếp request vào hàng đợi
- Xử lý request do luồng xử lý request đảm nhiệm, luồng này xử lý các request bao gồm cả những module vào ra
- Gửi kết quả cho client

SERVER TRONG HPT

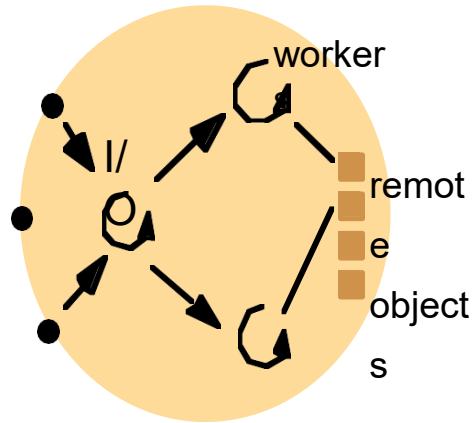
□ Mô hình server có điều phối (dispatcher)

- Bên cạnh các luồng để xử lý các công việc, tiến trình server có thêm 1 luồng chuyên dụng đảm nhiệm chức năng điều phối các request gọi là Dispatcher.
- Mỗi khi nhận được request luồng này sẽ phân tích request đó và gửi đến cho luồng phù hợp để xử lý

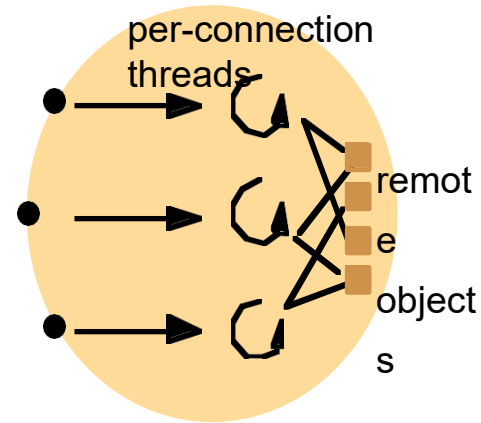
=> các luồng hoạt động hiệu quả hơn do mỗi luồng chuyên xử lý các công việc chuyên biệt do luồng điều phối phân loại (Dispatcher) và gửi đến



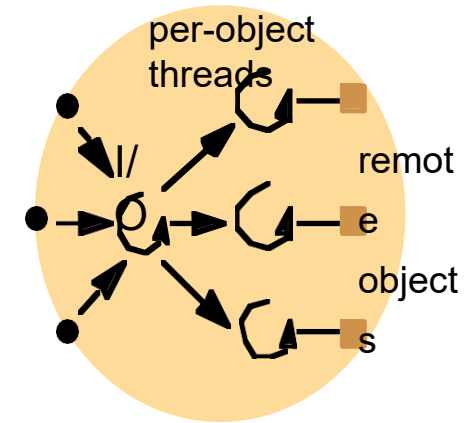
SERVER TRONG HPT



a. Thread-per-request



b. Thread-per-connection

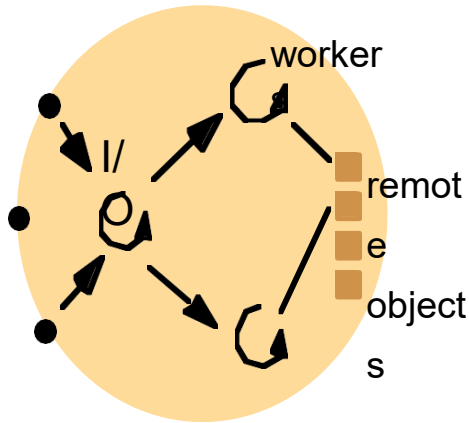


c. Thread-per-object

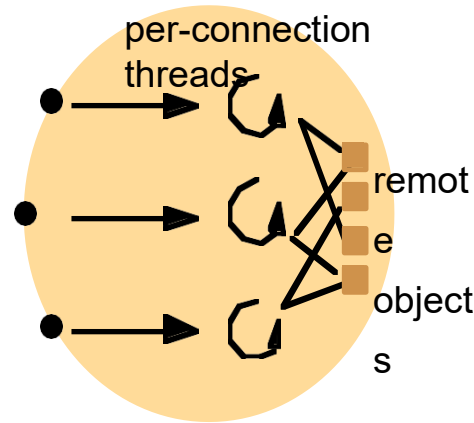
Ba kiến trúc tổ chức cơ bản cho server đa luồng

1. Thread-per-request (Luồng cho mỗi request)
2. Thread-per-connection (Luồng cho mỗi kết nối)
3. Thread-per-object (Luồng cho mỗi đối tượng)

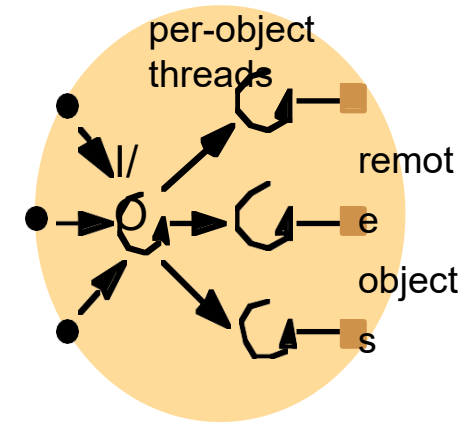
SERVER TRONG HPT



1. Thread-per-request



2. Thread-per-connection

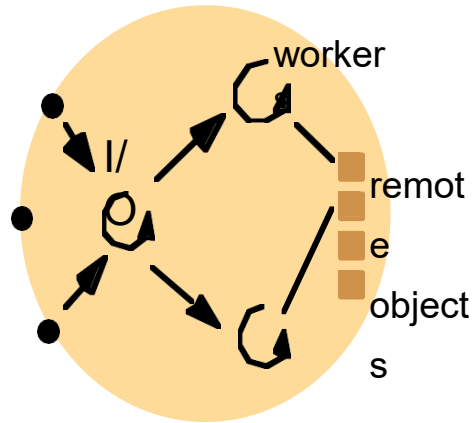


3. Thread-per-object

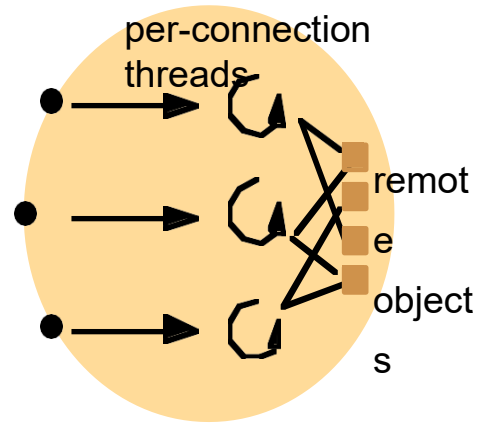
1. Thread-per-request (Luồng cho mỗi request)

- Tiến trình vào ra sinh ra 1 luồng worker mới cho mỗi request nhận được
- Mỗi luồng sẽ tự hủy sau khi hoàn thành nhiệm vụ
- Ưu điểm:
 - Không cần có hàng đợi vì mỗi y/c gửi đến đều được sinh ra 1 luồng và xử lý ngay
 - Băng thông có thể đạt mức tối đa vì có khả năng sinh không giới hạn các luồng của worker
- Nhược điểm: việc tạo, hủy luồng trong mỗi request nhận được, đặc biệt trong trường hợp có quá nhiều => giảm hiệu năng và làm tăng chi phí các hoạt động của hệ thống

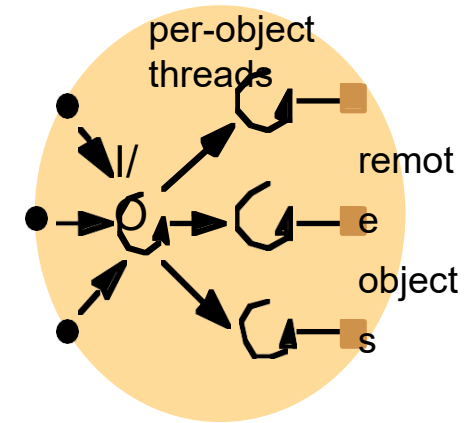
SERVER TRONG HPT



1. Thread-per-request



2. Thread-per-connection

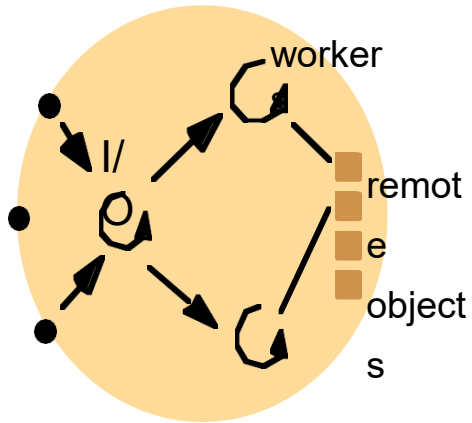


3. Thread-per-object

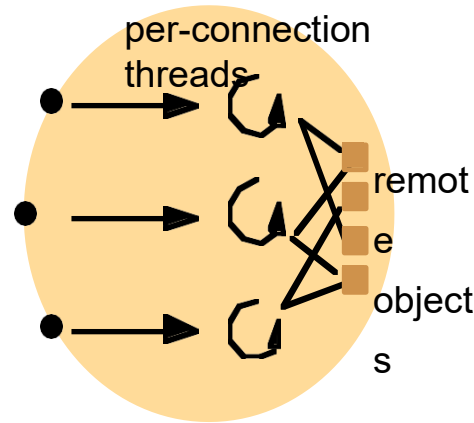
2. Thread-per-connection (Luồng cho mỗi kết nối)

- Gắn kết mỗi luồng với 1 kết nối.
- Khi 1 client gửi request kết nối đến với server, server sẽ tạo ra 1 luồng cho kết nối đó và hủy luồng nếu kết nối đó ngắt

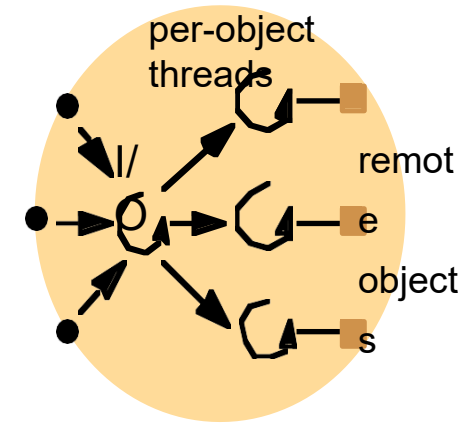
SERVER TRONG HPT



1. Thread-per-request



2. Thread-per-connection



3. Thread-per-object

3. Thread-per-object (Luồng cho mỗi đối tượng)

- Tiến trình gắn 1 luồng với 1 đối tượng từ xa (remote object) mà nó quản lý.
- Với kiến trúc này luồng I/O phải quản lý thêm hàng đợi cho mỗi object

Tuy nhiên, giống như Thread-per-connection, các client có thể bị phục vụ với độ trễ cao do 1 số luồng thì phục vụ quá nhiều request trong khi 1 số luồng khác có thể phục vụ ít/không tại cùng thời điểm

CLIENT TRONG HPT

Các kiến trúc tổ chức cơ bản trong Client đa luồng

- Tách biệt giao diện người dùng và xử lý:
 - Luồng xử lý hiển thị giao diện người dùng
 - Luồng xử lý bên trong

⇒ tăng tốc độ sử dụng hệ thống

⇒ che giấu được độ trễ của thời gian phục vụ.
- Tương tác với nhiều server khác nhau:
 - Ứng với mỗi server tiến trình client sinh ra 1 luồng để xử lý công việc tương ứng với server đó

⇒ Tăng tốc độ khi làm việc
- Che giấu các chi tiết cài đặt: phân tách 1 luồng để phục vụ giao diện người dùng, các luồng khác thực hiện các thao tác cài đặt cần thiết

BÀI TẬP

Lựa chọn một ứng dụng cụ thể bất kỳ sử dụng mô hình Client – Server. Trình bày giải pháp triển khai dưới dạng client-server đa luồng, trong đó lưu ý:

- Mô tả ứng dụng và các chức năng chính.
- Chỉ rõ lựa chọn phương thức tổ chức client, server nào; vẽ sơ đồ luồng và giải thích lý do lựa chọn.
- Bài làm:
 - Trình bày tối thiểu 1 trang và tối đa 2 trang A4
 - Thời gian làm: 60p
 - Nộp bài: Canvas

BÀI TẬP

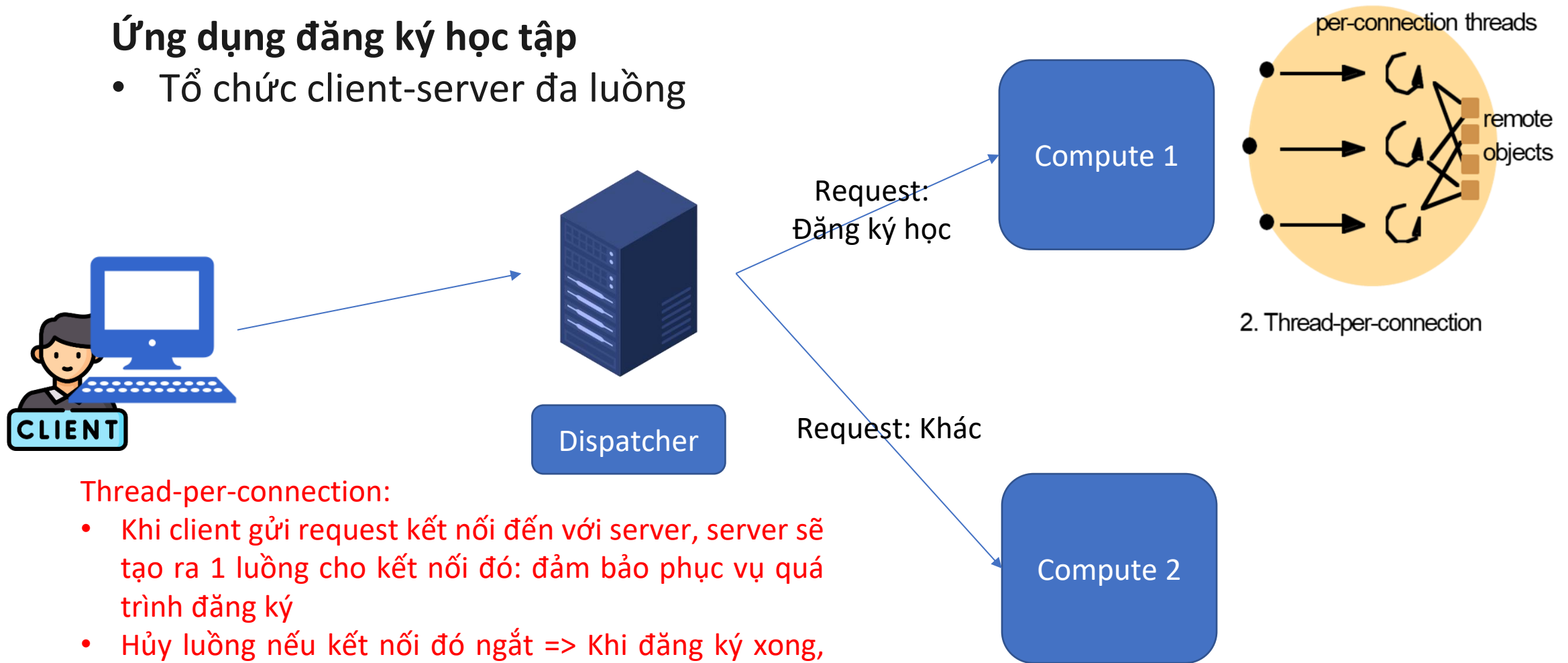
Ứng dụng Cổng thông tin sinh viên

- Chức năng chính
 - Đăng nhập
 - Thanh toán học phí
 - Cập nhật, tra cứu thông tin,...
 - Đăng ký học
- Đối tượng: Sinh viên các khoá
 - Năm nhất
 - Năm hai
 - Năm ba
 - Năm tư

BÀI TẬP

Ứng dụng đăng ký học tập

- Tổ chức client-server đa luồng



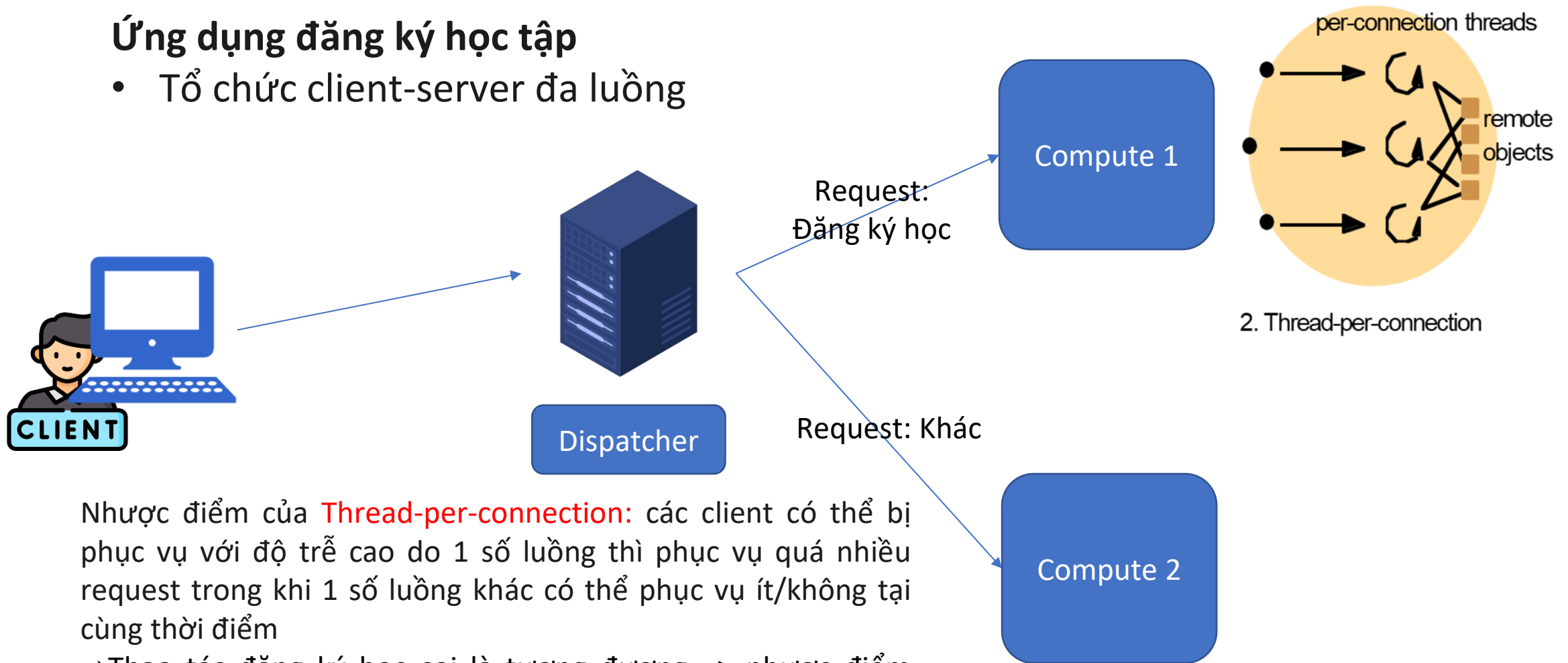
Thread-per-connection:

- Khi client gửi request kết nối đến với server, server sẽ tạo ra 1 luồng cho kết nối đó: đảm bảo phục vụ quá trình đăng ký
- Hủy luồng nếu kết nối đó ngắt => Khi đăng ký xong, hoặc đặt bộ đếm timer

BÀI TẬP

Ứng dụng đăng ký học tập

- Tổ chức client-server đa luồng



Nhược điểm của **Thread-per-connection**: các client có thể bị phục vụ với độ trễ cao do 1 số luồng thì phục vụ quá nhiều request trong khi 1 số luồng khác có thể phục vụ ít/không tại cùng thời điểm

⇒ Thao tác đăng ký học coi là tương đương => nhược điểm được khắc phục